Exercício 1

O *Conway's Game of Life* é um exemplo de um autómato celular. Neste problema modificam-se as regras do autómato de forma a usar um espaço de estados finito.

Regras do autómato

1. O espaço de estados é definido por uma grelha de células booleanas (morta = `0` / viva = `1`) de dimensão

   $$N \times N$$

   com $N > 16$, identificadas por índices

   $$(i, j) \in \{1, \ldots, N\}.$$

   Estas N^2 células são aqui referidas como **"normais"**.

2. Inicialmente, todas as células normais estão mortas, exceto as células com $i, j \leq 3$, que estão vivas. Um estado onde todas as células normais estão mortas é um **"estado de erro"**.

3. Adicionalmente, existem:

   A. $2N + 1$ **"células iniciais"** que correspondem a um dos índices, i ou j, ser zero. As células iniciais têm valores constantes que, no estado inicial, são gerados aleatoriamente com probabilidade $1/2$ de estarem vivas.

   B. $2N + 3$ **"células finais"** que não são iniciais e em que um dos seus índices tem o valor $N + 1$. As células finais têm valor constantemente 0, isto é, estão mortas.

4. As células normais do autómato modificam o estado de acordo com a regra **"B3|S23"**: a célula nasce (passa de 0 para 1) se tem exatamente 3 vizinhos vivos; sobrevive (mantém-se viva) se o número de vizinhos vivos é 2 ou 3; caso contrário, morre ou continua morta.

5. As células (i_0, j_0) e (i_1, j_1) são vizinhas se e só se

   $$\max\big(|i_0 - i_1|,\ |j_0 - j_1|\big) = 1.$$

Pretende-se

1. Construir uma máquina de estados finita (FSM) que represente este autómato.

Este problema tem duas interpretações possíveis para as transições da FSM:

   A. Em cada transição do sistema, uma e só uma célula normal transita de estado.
   B. Em cada transição do sistema, todas as células normais transitam de estado simultaneamente.

   Ambas as opções devem ser contempladas e a relação de transição selecionada é determinada pelo valor de um parâmetro definido no código.

2. Provar (ou obter contraexemplos) para as seguintes propriedades:

   A. Nunca se alcança o estado de erro.
   B. Nenhuma célula normal está permanentemente morta ou permanentemente viva.

In [ ]:
```
import marimo as mo
import random
import time
from z3 import Bool, BoolVal, Solver, Sum, If, Or, And, Not, Implies, PbEq,
is_true, sat
```

In [ ]:
```
class Box:
    def __init__(self, n, modo="SYNC", celulas=None, iniciais=None,
finais=None, seed=None):
        if n <= 16:
            raise ValueError("N precisa ser > 16")
        if modo not in ("SYNC", "ASYNC"):
            raise ValueError("modo tem de ser 'SYNC' ou 'ASYNC'")
        self.n = n
        self.modo = modo
        self.tamanho = n * n
        self.rng = random.Random(seed)

        # células normais vivas: bloco 3x3
        self.vivas = set()
        if celulas is None:
            celulas = {(i, j): True for i in range(1, 4) for j in range(1,
4)}
        for (i, j), v in celulas.items():
            self.add(i, j, v)

        # células iniciais: i = 0 ou j = 0 (constantes aleatórias)
        if iniciais is None:
            iniciais = {(i, j): self.rng.random() < 0.5
                        for i in range(self.n + 1)
                        for j in range(self.n + 1)
                        if i == 0 or j == 0}
        self.iniciais = iniciais

        # células finais: i = N+1 ou j = N+1 (constantes a 0)
        if finais is None:
            finais = {(i, j): False
                      for i in range(0, self.n + 2)
                      for j in range(0, self.n + 2)
```

```
                        if i == self.n + 1 or j == self.n + 1}
        self.finais = finais

        assert len(self.iniciais) == 2 * n + 1
        assert len(self.finais) == 2 * n + 3

    # ---------- simulação ----------
    def add(self, i, j, vale=True):
        if not (1 <= i <= self.n and 1 <= j <= self.n):
            raise ValueError(f"Coordenada ({i},{j}) fora da grelha
{self.n}x{self.n}")
        if vale:
            self.vivas.add((i, j))
        else:
            self.vivas.discard((i, j))

    def val(self, i, j):
        if (i, j) in self.finais:
            return self.finais[(i, j)]
        if (i, j) in self.iniciais:
            return self.iniciais[(i, j)]
        return (i, j) in self.vivas

    def vizinhos_vivos(self, i, j):
        return sum(self.val(i + a, j + b)
                   for a in (-1, 0, 1)
                   for b in (-1, 0, 1)
                   if (a, b) != (0, 0))
# B3/S23

    def regra(self, i, j):
        k = self.vizinhos_vivos(i, j)
        return k == 3 or ((i, j) in self.vivas and k == 2)



    def passo(self, celula=None):
        if self.modo == "SYNC":
            # todas as células normais transitam em simultâneo
            novo = {(i, j)
                    for i in range(1, self.n + 1)
                    for j in range(1, self.n + 1)
                    if self.regra(i, j)}
            self.vivas = novo
        else:  # ASYNC: só uma célula transita
            if celula is  None:
                i = self.rng.randint(1, self.n)
                j = self.rng.randint(1, self.n)
            else:
                i, j = celula
            self.add(i, j, self.regra(i, j))



    def erro(self):
        return len(self.vivas) == 0
```

```
    def __str__(self):
    # Usa self.val(i, j) para consultar vivas, iniciais e finais
    # range(0, self.n + 2) percorre de 0 até N+1
      return "\n".join(
        "".join("#" if self.val(i, j) else "."
                for j in range(0, self.n + 2))
        for i in range(0, self.n + 2)
    )

    # ---------- FSM em Z3 =----------
    def modelo(self, K):

        normais = [(i, j) for i in range(1, self.n + 1) for j in range(1,
self.n + 1)]
        ini = {c: Bool(f"i_{c[0]}_{c[1]}") for c in self.iniciais}   #
livres e constantes
         #ini = {}
          #for c in self.iniciais:        # c = (0,0), (0,1), ..., (N,0)
           #ini[c] = Bool(f"i_{c[0]}_{c[1]}")


        st = [{c: Bool(f"s{t}_{c[0]}_{c[1]}") for c in normais} for t in
range(K + 1)]
        #   st = []
           #for t in range(K + 1):          # K+1 fotografias
         #   foto = {}
           #  for c in normais:           # N² células, de (1,1) a (N,N)
            #     foto[c] = Bool(f"s{t}_{c[0]}_{c[1]}")
               # st.append(foto)
        pk = [{c: Bool(f"p{t}_{c[0]}_{c[1]}") for c in normais} for t in
range(K)]

        def v(t, c): # A função v(t, c) responde a uma pergunta: “a célula
c está viva no passo t
            if c in st[t]:
                return st[t][c]
            if c in ini:
                return ini[c]
            return BoolVal(self.finais.get(c, False))

        def regra(t, c):
            i, j = c
            k = Sum([If(v(t, (i + a, j + b)), 1, 0)
                     for a in (-1, 0, 1) for b in (-1, 0, 1) if (a, b) !=
(0, 0)])
            return Or(k == 3, And(st[t][c], k == 2))

        F = [st[0][c] == BoolVal(c[0] <= 3 and c[1] <= 3) for c in normais]
        for t in range(K):
            if self.modo == "SYNC":
                F += [st[t + 1][c] == regra(t, c) for c in normais]
            else:
                F.append(PbEq([(pk[t][c], 1) for c in normais], 1))
                F += [st[t + 1][c] == If(pk[t][c], regra(t, c), st[t][c])
for c in normais]
```

```
                #Para cada célula c, o If(condição, A, B) funciona assim:

                #se pk[t][c] é verdadeiro (foi a escolhida) → o valor no
passo seguinte é regra(t, c) (aplica B3/S23);
                #senão → o valor é st[t][c] (mantém o que tinha).
        return F, st, pk, ini, normais

    # ---------- Propriedade 1: G !erro ----------
    def verifica_erro(self, K):
        F, st, pk, ini, normais = self.modelo(K)
        s = Solver()
        s.add(F)
        s.add(Or([And([Not(x) for x in st[t].values()]) for t in range(K +
1)]))
        if s.check() != sat:
         return {
           "violada": False,
           "msg": f"Sem erro até profundidade {K} (BMC, não é prova total)"
    }

        m = s.model()
        return {
         "violada": True,
         "msg": f"Erro (extinção) alcançado na profundidade {K}!",
         "iniciais": self._ini(m, ini),
         "picks": self._picks(m, pk, normais, K)}


    # ---------- Propriedade 2: Nenhuma célula permanentemente morta ou
viva ----------
    def verifica_permanente(self, K, viva=False):
        F, st, pk, ini, normais = self.modelo(K)

        s = Solver()
        s.add(F)
        L = [Bool(f"L{l}") for l in range(K)]
        s.add(PbEq([(x, 1) for x in L], 1))


        # L[l]: indica o instante l onde o lasso fecha
        L = [Bool(f"L{l}") for l in range(K)]
        s.add(PbEq([(x, 1) for x in L], 1))

        for l in range(K):
            s.add(Implies(L[l], And([st[K][c] == st[l][c] for c in
normais])))

        no_ciclo = lambda t: Or(L[:t + 1])

        # Filtra por célula permanentemente viva ou morta consoante o
parâmetro viva
        para_sempre = lambda c: And([Implies(no_ciclo(t), st[t][c] if viva
else Not(st[t][c])) for t in range(K)])

        s.add(Or([para_sempre(c) for c in normais]))
```

```
        estado_str = "viva" if viva else "morta"
        if s.check() == sat:
            m = s.model()
            return {
                "violada": True,
                "msg": f"Propriedade 2 violada: Existe pelo menos uma
célula permanentemente {estado_str} no ciclo (até K={K}).",
                "iniciais": self._ini(m, ini),
                "picks": self._picks(m, pk, normais, K)
            }
        else:
            return {
                "violada": False,
                "msg": f"Nenhum lasso de comprimento ≤ {K} contém células
sempre {estado_str}."
            }

             # 1. Condição para a célula c estar permanentemente morta no
ciclo
        #morta_no_ciclo = lambda c: And([Implies(no_ciclo(t), Not(st[t]
[c])) for t in range(K)])

        # 2. Condição para a célula c estar permanentemente viva no ciclo
        #viva_no_ciclo  = lambda c: And([Implies(no_ciclo(t), st[t][c]) for
t in range(K)])

        # 3. Uma célula é estática se cumprir QUALQUER UMA das duas
        #celula_estatica = lambda c: Or(morta_no_ciclo(c),
viva_no_ciclo(c))

        # 4. Adicionar ao Z3 usando 'celula_estatica'
    # ---------- extrair contraexemplo e fazer replay ----------
    def _ini(self, m, ini):
        return {c: is_true(m.eval(x, model_completion=True)) for c, x in
ini.items()}

    def _picks(self, m, pk, normais, K):
        if self.modo == "SYNC":
            return None
        return [next(c for c in normais if is_true(m.eval(pk[t][c],
model_completion=True)))for t in range(K)]
```

Exercício 1: Game of Life com espaço de estados finito

Problema

Autómato celular na grelha $N \times N$ ($N > 16$) com regra B3/S23 e vizinhança de Moore ($\max(|i_0 - i_1|, |j_0 - j_1|) = 1$). A grelha completa tem índices $0..N+1$:

Tipo	Índices	Nº	Comportamento
normais	$1..N \times 1..N$	N^2	evoluem pela regra B3/S23
iniciais	$i = 0$ ou $j = 0$	$2N + 1$	constantes, valor aleatório (prob. 1/2)
finais	$i = N + 1$ ou $j = N + 1$	$2N + 3$	constantes a 0

Estado inicial: células normais com $i, j \leq 3$ vivas, as restantes mortas. Estado de erro: todas as células normais mortas.

Abordagem

- **FSM:** o estado é o vetor das N^2 células normais. As células iniciais não mudam no tempo, por isso são modeladas como **variáveis booleanas livres**. O solver verifica as propriedades para todas as 2^{2N+1} fronteiras possíveis de uma só vez.
- **Transição** (parâmetro `modo`):
  - `SYNC`: todas as células normais transitam em simultâneo.
  - `ASYNC`: uma e só uma célula transita por passo. A escolha é uma variável livre `pick[t][c]`.
- **Propriedades** (verificadas com Z3 por *Bounded Model Checking*):
  1. $G\,\neg\text{erro}$: nunca se alcança o estado de erro.
  2. Nenhuma célula normal está permanentemente morta nem permanentemente viva. A negação ($\exists c.\ FG\,\text{morta} \vee FG\,\text{viva}$) é procurada como um *lasso* (o estado no passo K repete um estado anterior).
- **Simulação:** a classe `Box` também simula o autómato. Usa-se para validar o modelo Z3 (repetindo os contraexemplos) e para testar a regra com padrões conhecidos.

In [ ]:
```python
def testar_e_mostrar_erro(n=17, K=12, modo="ASYNC"):
    print(f"=== VERIFICAÇÃO DE ERRO (Propriedade 1) ===")
    print(f"Grelha: {n}x{n} | Modo: {modo} | Passos (K): {K}\n")

    # 1. Instanciar o modelo
    box = Box(n=n, modo=modo, seed=42)

    # 2. Executar a verificação formal com Z3
    resultado = box.verifica_erro(K)

    if not resultado["violada"]:
        print(f"✅ {resultado['msg']}")
```

```
        return

    print(f"❌ {resultado['msg']}\n")
    print("=" * 45)
    print(" REPRODUÇÃO DO CONTRA-EXEMPLO PASSO A PASSO ")
    print("=" * 45 + "\n")

    # 3. Instanciar simulação local com as bordas 'iniciais' descobertas
pelo Z3
    simulacao = Box(n=n, modo=modo, iniciais=resultado["iniciais"])

    # Exibir estado t = 0
    print(f"--- Foto t = 0 (Início) ---")
    print(simulacao)
    print(f"Células vivas: {len(simulacao.vivas)}\n")

    picks = resultado.get("picks", [])

    # 4. Avançar passo a passo no tempo
    for t in range(K):
        if modo == "SYNC":
            simulacao.passo()
            print(f"--- Foto t = {t + 1} (Passo Síncrono) ---")
        else:
            celula_sorteada = picks[t]
            simulacao.passo(celula=celula_sorteada)
            print(f"--- Foto t = {t + 1} (ASYNC: Z3 atualizou a célula
{celula_sorteada}) ---")

        print(simulacao)
        vivas_atuais = len(simulacao.vivas)
        print(f"Células vivas: {vivas_atuais}")

        if simulacao.erro():
            print("\n⚠️  [ESTADO DE ERRO ALCANÇADO: EXTINÇÃO TOTAL!]")
            print("Todas as células normais morreram.")
            break
        print()

# Executar o teste no modo ASYNC
testar_e_mostrar_erro(n=17, K=9, modo="SYNC")
```

```
=== VERIFICAÇÃO DE ERRO (Propriedade 1) ===
Grelha: 17x17 | Modo: SYNC | Passos (K): 9

❌ Erro (extinção) alcançado na profundidade 9!

================================================
 REPRODUÇÃO DO CONTRA-EXEMPLO PASSO A PASSO
================================================

--- Foto t = 0 (Início) ---
.##..##...#..#..##.
####..............
####..............
.###..............
.................
#.................
#.................
.................
#.................
.................
#.................
.................
#.................
.................
.................
#.................
.................
.................
.................
Células vivas: 9

--- Foto t = 1 (Passo Síncrono) ---
.##..##...#..#..##.
#...#.............
#...#.............
...#..............
.##...............
#.................
#.................
.................
#.................
.................
#.................
.................
#.................
.................
.................
#.................
.................
.................
.................
Células vivas: 5

--- Foto t = 2 (Passo Síncrono) ---
.##..##...#..#..##.
#..##.............
```

```
#..##.............
.###..............
.##...............
#..................
#..................
..................
#..................
..................
#..................
..................
#..................
..................
..................
#..................
..................
..................
..................
Células vivas: 9

--- Foto t = 3 (Passo Síncrono) ---
.##..##...#..#..##.
#..................
#..................
....#..............
...#...............
#..................
#..................
..................
#..................
..................
#..................
..................
#..................
..................
..................
#..................
..................
..................
..................
Células vivas: 2

--- Foto t = 4 (Passo Síncrono) ---
.##..##...#..#..##.
#..................
#..................
..................
..................
#..................
#..................
..................
#..................
..................
#..................
..................
#..................
..................
```

```
..................
#.................
..................
..................
..................
Células vivas: 0

⚠️  [ESTADO DE ERRO ALCANÇADO: EXTINÇÃO TOTAL!]
Todas as células normais morreram.
```

In [ ]:

```python
def testar_e_mostrar_permanente(n=17, K=5, modo="SYNC"):
    print(f"=== VERIFICAÇÃO DE CÉLULAS PERMANENTES (Propriedade 2) ===")
    print(f"Grelha: {n}x{n} | Modo: {modo} | Passos (K): {K}\n")

    box = Box(n=n, modo=modo, seed=42)

    # Testar ambos os estados: permanentemente mortas e permanentemente vivas
    for estado_viva in [False, True]:
        tipo_str = "VIVAS" if estado_viva else "MORTAS"
        print(f"--- A testar células permanentemente {tipo_str} ---")

        resultado = box.verifica_permanente(K, viva=estado_viva)
        msg = resultado.get("msg", "Sem mensagem.")

        if not resultado.get("violada", False):
            print(f"✅ {msg}\n")
            continue

        print(f"❌ {msg}\n")
        print("=" * 45)
        print(f" REPRODUÇÃO DO CONTRA-EXEMPLO ({tipo_str}) ")
        print("=" * 45 + "\n")

        # Instanciar a simulação com as bordas do contra-exemplo
        simulacao = Box(n=n, modo=modo, iniciais=resultado["iniciais"])
        picks = resultado.get("picks", [])

        print(f"--- Foto t = 0 (Início) ---")
        print(simulacao)
        print(f"Células vivas: {len(simulacao.vivas)}\n")

        # Reprodução passo a passo
        for t in range(K):
            if modo == "SYNC":
                simulacao.passo()
                print(f"--- Foto t = {t + 1} (SYNC) ---")
            else:
                celula_sorteada = picks[t]
                simulacao.passo(celula=celula_sorteada)
                print(f"--- Foto t = {t + 1} (ASYNC: Célula {celula_sorteada}) ---")

            print(simulacao)
            print(f"Células vivas: {len(simulacao.vivas)}\n")
```

```
        print("=" * 45 + "\n")

# Executar o teste
testar_e_mostrar_permanente(n=17, K=5, modo="SYNC")
```

```
=== VERIFICAÇÃO DE CÉLULAS PERMANENTES (Propriedade 2) ===
Grelha: 17x17 | Modo: SYNC | Passos (K): 5

--- A testar células permanentemente MORTAS ---
❌ Propriedade 2 violada: Existe pelo menos uma célula permanentemente morta
no ciclo (até K=5).

==============================================
 REPRODUÇÃO DO CONTRA-EXEMPLO (MORTAS)
==============================================

--- Foto t = 0 (Início) ---
..#................
.###...............
####...............
.###...............
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
Células vivas: 9

--- Foto t = 1 (SYNC) ---
..#................
..................
#...#..............
...#...............
..#................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
Células vivas: 3

--- Foto t = 2 (SYNC) ---
```

```
..#................
..................
#.................
...#..............
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
Células vivas: 1

--- Foto t = 3 (SYNC) ---
..#................
..................
#.................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
Células vivas: 0

--- Foto t = 4 (SYNC) ---
..#................
..................
#.................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
```

```
..................
..................
..................
..................
..................
..................
..................
Células vivas: 0

--- Foto t = 5 (SYNC) ---
..#...............
..................
#.................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
..................
Células vivas: 0

================================================

--- A testar células permanentemente VIVAS ---
❌ Propriedade 2 violada: Existe pelo menos uma célula permanentemente viva
no ciclo (até K=5).

================================================
 REPRODUÇÃO DO CONTRA-EXEMPLO (VIVAS)
================================================

--- Foto t = 0 (Início) ---
..#.....###........
.###...............
####...............
.###...............
..................
..................
#.................
..................
..................
..................
..................
#.................
#.................
#.................
```

```
#.................
#.................
#.................
..................
..................
Células vivas: 9

--- Foto t = 1 (SYNC) ---
..#.....###........
.........#.........
#...#..............
...#...............
..#................
..................
#.................
..................
..................
..................
..................
#.................
##................
##................
##................
##................
#.................
..................
..................
Células vivas: 8

--- Foto t = 2 (SYNC) ---
..#.....###........
........###........
#.................
...#...............
..................
..................
#.................
..................
..................
..................
..................
##................
#.................
#.#................
#.#................
#.................
##................
..................
..................
Células vivas: 8

--- Foto t = 3 (SYNC) ---
..#.....###........
........#.#........
#........#.........
..................
```

```
..................
..................
#.................
..................
..................
..................
..................
##................
#.................
#.................
#.................
#.................
##................
..................
..................
Células vivas: 5

--- Foto t = 4 (SYNC) ---
..#.....###.......
........#.#.......
#........#........
..................
..................
..................
#.................
..................
..................
..................
..................
##................
#.................
##................
##................
#.................
##................
..................
..................
Células vivas: 7

--- Foto t = 5 (SYNC) ---
..#.....###.......
........#.#.......
#........#........
..................
..................
..................
#.................
..................
..................
..................
..................
##................
#.................
#.................
#.................
#.................
```

```
##.................
...................
...................
Células vivas: 5

============================================
```

In [ ]:

mo.md(r"""

Conclusões

🔍 1. Análise das Propriedades

- **Propriedade 1 — Estado de Erro / Extinção ($G\neg\text{erro}$)**
  - **Resultado: ❌ VIOLADA**
  - **Análise:** O Z3 encontrou uma configuração de valores nas bordas iniciais (e uma sequência específica de atualização no modo `ASYNC`) que levou à extinção total das células normais ao fim de $K = 5$ passos.
- **Propriedade 2 — Ausência de Células Permanentes**
  - **Resultado: ❌ VIOLADA**
  - **Análise:** Numa grelha de 17×17 com profundidade $K = 5$, as células localizadas nos cantos mais afastados do bloco 3×3 inicial não têm tempo de propagação suficiente para mudar de estado, permanecendo mortas durante todo o ciclo (*lasso*).

🔄 2. Comparação de Modos: SYNC vs. ASYNC

Modo	Mecanismo de Transição	Impacto na Verificação Formal (Z3)
`SYNC`	Todas as células transitam em simultâneo a cada passo.	**Mais rápido:** Transição determinística global, sem necessidade de escolhas de ordenação.
`ASYNC`	Apenas uma célula transita por passo t.	**Mais complexo:** O Z3 precisa de resolver variáveis de seleção (pk) a cada instante, aumentando o espaço de busca e o tempo de execução.

🎯 3. Limitações do *Bounded Model Checking* (BMC)

- **Validade Limitada ao Limite K:** O BMC é uma técnica focada em encontrar contra-exemplos e falhas num horizonte finito K.
- **Ausência de Prova Indutiva:** Um resultado de "sem erro até $K = 5$" garante a propriedade exclusivamente até ao passo 5, não constituindo uma prova de segurança para o tempo infinito ($t \to \infty$).

Utilização de LLM

https://gemini.google.com/app/f3c33809e4179d8b "\"\")